

# Magic2

## Architecture and Developer Notes

Internal Document — Not for End Users

---

Jean Thierry-Mieg, Danielle Thierry-Mieg, Greg Boratyn

National Library of Medicine, National Institutes of Health  
8600 Rockville Pike, Bethesda MD 20894, USA

`mieg@ncbi.nlm.nih.gov`

Source code: <https://github.com/NLM-DIR/Magic2>

National Library of Medicine · National Institutes of Health

*This document describes internal implementation details. For user-facing documentation see the Magic2 User Guide.*

## Contents

---

## 1 Purpose of This Document

---

This document is intended for developers working on the Magic2 source code, not for end users. It describes the internal pipeline architecture, the parallelism model, the NUMA-aware process model, the GPU code path, and other implementation details that are not relevant to running alignments but are essential for understanding, debugging, or extending the code.

For user-facing documentation, see the Magic2 User Guide ([wsa\\_doc/magic2\\_userguide.pdf](#)).

## 2 Pipeline Architecture

---

### 2.1 Overview

Magic2 is structured as a deep pipeline of independent processing layers. Data flows through the pipeline as discrete blocks; all layers execute concurrently and self-balance their workloads. This design is inspired by the channel and goroutine model of the Go programming language, implemented in C using a custom channel library (see Section ??).

The key architectural properties are:

- All pipeline layers run simultaneously. There is no master thread dispatching work; each layer pulls from its input channel and pushes to its output channel independently.
- Work is divided into *data blocks* of at most **bMax** megabases each. The number of blocks simultaneously in flight is **nBlocks**.
- Each layer contains **nAgents** parallel agents. All agents within a layer are interchangeable and process whichever block arrives next.
- RAM consumption is controlled by a *dropper* mechanism at the head of the pipeline that withholds new blocks when memory pressure is high, rather than by pre-allocating a fixed buffer.
- Input size is unbounded: blocks flow in and out continuously, so a run of 100 Gbases is handled identically to a run of 1 Gbase.

### 2.2 Pipeline layers

The main alignment pipeline consists of the following layers, in order:

| Layer | Name     | Function                                                                               |
|-------|----------|----------------------------------------------------------------------------------------|
| 1     | Reader   | Reads and decompresses input files or SRA streams; emits read blocks.                  |
| 2     | Seeder   | Extracts seeds from reads; looks up seeds in the index (CPU or GPU).                   |
| 3     | Sorter   | Radix-sorts seed hits to group candidates by target position.                          |
| 4     | Extender | Extends seed hits to full-length alignments; resolves introns on the CPU.              |
| 5     | Filter   | Applies score, length, and error-rate filters; resolves multi-mappers.                 |
| 6     | Writer   | Formats and writes BAM, <code>.hits</code> , wiggles, intron tables, and count tables. |

**Note:** The GPU accelerates layers 2 and 3 (seeding and sorting). Layer 4 (extension), which requires semantically complex splice-junction logic, remains on the CPU in the current implementation. See Section ??.

## 2.3 Key internal parameters

| Variable                 | Flag                       | Default                     | Meaning                                   |
|--------------------------|----------------------------|-----------------------------|-------------------------------------------|
| <code>nAgents</code>     | <code>--nA</code>          | $3/2 \times n_{\text{CPU}}$ | Agents per pipeline layer.                |
| <code>nBlocks</code>     | <code>--nB</code>          | $2 \times n_{\text{CPU}}$   | Simultaneous blocks in flight.            |
| <code>bMax</code>        | <code>--bMax</code>        | 3 MB                        | Maximum megabases per block.              |
| <code>max_threads</code> | <code>--max_threads</code> | hardware-derived            | Hard cap on UNIX threads.                 |
| <code>tStep</code>       | <code>--step</code>        | 6 (large genomes)           | Seed step used during index construction. |
| <code>rStep</code>       | <code>--rStep</code>       | <code>tStep</code> /2       | Seed step used during read alignment.     |
| <code>NN</code>          | <code>--NN</code>          | 16                          | Number of seed file partitions.           |

**Warning:** `max_threads` must not be set below 64. The channel architecture requires a minimum level of concurrency; an artificially low thread cap will cause the pipeline to stall waiting for virtual threads that cannot be scheduled.

## 3 The Channel Library

### 3.1 Motivation

Standard C parallelism (pthreads, OpenMP) is based on shared memory and explicit locking. Magic2 instead uses a message-passing model inspired by Go channels: data blocks are passed between layers through typed channels; no layer directly reads or writes another layer’s memory.

This design was chosen because:

- It eliminates entire classes of race conditions: a block is owned by exactly one layer at a time.

- It makes the pipeline self-documenting: the channel topology is the architecture diagram.
- It scales naturally: adding agents to a layer requires no changes to the surrounding code.

### 3.2 Channel semantics

Each channel is a bounded FIFO queue. A layer blocks on a read if the channel is empty and blocks on a write if the channel is full. The dropper at the head of the pipeline monitors RAM usage and delays injection of new blocks when memory is constrained, providing back-pressure throughout the system without any layer needing to be aware of memory state.

### 3.3 Virtual threads

Each agent and each channel endpoint runs in its own virtual thread. Virtual threads are lightweight: the system may have thousands of them scheduled over a smaller number of UNIX threads. The `max.threads` parameter caps the number of UNIX threads, not the number of virtual threads.

## 4 NUMA-Aware Process Model

---

### 4.1 NUMA detection

On startup, Magic2 counts the number of NUMA nodes by scanning `/sys/devices/system/node`. If the directory is absent (non-NUMA kernel, container environment) or contains only one node, the count is treated as 1 and no re-spawning occurs.

```
static int count_numa_nodes (void)
{
    DIR *d = opendir ("/sys/devices/system/node") ;
    if (!d) return 1 ;
    int count = 0, dummy ;
    struct dirent *e ;
    while ((e = readdir (d)))
        if (sscanf (e->d_name, "node%d", &dummy) == 1)
            count++ ;
    closedir (d) ;
    return count > 0 ? count : 1 ;
}
```

### 4.2 Re-spawn mechanism

When more than one NUMA node is detected, Magic2 identifies the least loaded node and re-spawns itself using `numactl --cpunodebind=N --membind=N`, where N is the chosen node. This ensures that all memory allocations for the pipeline are local to the CPUs that will process them, avoiding the latency penalty of cross-node memory access.

The re-spawned child receives the flag `--numactl` in its argument list. This flag signals that re-spawning has already occurred and suppresses a second re-spawn, preventing infinite recursion.

### 4.3 Implications for debugging

When re-spawning occurs, a debugger attached to the parent process will not follow execution into the child. Breakpoints set before the `exec` call will never be hit.

Two command-line flags are provided to work around this:

| Flag                   | Effect                                                                                                                                                                                                     |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--debug</code>   | Suppresses re-spawning and forces <code>--nAgents 1 --nBlocks 1</code> , giving a fully serial execution path. Verbose diagnostic messages are enabled automatically. Use this for step-by-step debugging. |
| <code>--numactl</code> | Suppresses re-spawning only. Full parallelism is retained. Use this when profiling with <code>perf</code> or <code>vtune</code> , or when you need breakpoints but also need realistic concurrency.        |

Typical gdb sessions:

```
# Fully serial, easiest to step through:
gdb --args magic2 --debug -x IDX -i test.fa -o /tmp/dbg/

# Breakpoints work, full parallelism retained:
gdb --args magic2 --numactl -x IDX -i test.fa -o /tmp/dbg/
```

**Note:** On single-node machines, in containers, and on non-NUMA kernels, re-spawning never occurs and breakpoints work without any special flag.

## 5 GPU Acceleration

### 5.1 Scope

The GPU accelerates the two most computationally intensive layers of the pipeline: seed extraction and lookup (layer 2) and radix sorting of seed hits (layer 3). These operations are embarrassingly parallel and map well onto GPU execution models.

Layer 4 (alignment extension) remains on the CPU. Extension requires splice-junction logic, gap penalties, and intron model evaluation that are semantically complex and do not parallelise efficiently on a GPU in their current form. This may change in future versions.

### 5.2 Internal binary datasets

Several index data structures have GPU-specific variants that differ in memory layout from the CPU versions. These are generated at index construction time when a GPU compiler is available. The flag `--noJump`, which controls whether jumper entries are inserted into the index, is a compile-time internal switch relevant only to the GPU build; it is not exposed in the user-facing or developer help.

### 5.3 Fallback

If no CUDA-capable GPU is present at runtime, Magic2 falls back silently to the CPU path for all layers. No flag is needed.

## 6 Seed Parameters

---

### 6.1 Index construction step (`tStep`)

During index construction, a seed is extracted from the target every `tStep` bases. `tStep` is the *maximum* distance between consecutive seeds; because the best seed among the next `tStep` words is selected, the average distance is `tStep/2`.

The default is 1 for targets smaller than 1 Mb and 6 for larger genomes. Values of 8–10 reduce index RAM and construction time at a modest sensitivity cost; values of 1–4 increase sensitivity at higher RAM and CPU cost.

### 6.2 Alignment step (`rStep`)

During read alignment, seeds are extracted from the query every `rStep` bases. By default `rStep = tStep / 2`.

`rStep` is exposed in `--devhelp` as `--rStep` for benchmarking purposes. It should not be changed in production use.

### 6.3 Seed file partitioning (`NN`)

The seed index is split into `NN` partitions to allow parallel lookup. The allowed values are powers of 2 from 1 to 64. When `seedLength` exceeds 16, the minimum required value of `NN` is  $4^{(\text{seedLength}-16)}$ ; this minimum is always enforced automatically.

## 7 The TSF Format

---

### 7.1 Overview

TSF (Tag-Sample Format) is the internal tabular format used by Magic2 for all multi-sample output: gene expression counts, intron support tables, poly-A addition sites, QC metrics, and coverage summaries.

### 7.2 Line structure

Each line contains four tab-separated fields:

| Column | Content  | Notes                                                                                                      |
|--------|----------|------------------------------------------------------------------------------------------------------------|
| 1      | Run name | Identifies the sample or run.                                                                              |
| 2      | Tag      | Feature identifier (gene name, intron coordinates, etc.).                                                  |
| 3      | Format   | Type code string describing the data columns (e.g. <b>i</b> = integer, <b>f</b> = float, <b>t</b> = text). |
| 4+     | Data     | One or more data values matching the format code.                                                          |

Lines need not be sorted. TSF files from different runs can be concatenated directly; the TSF library merges them by joining on column 1, column 2, or both.

### 7.3 Type codes

The format field (column 3) is a string of single-character type codes, one per data column:

| Code     | Type    |
|----------|---------|
| <b>i</b> | Integer |
| <b>f</b> | Float   |
| <b>t</b> | Text    |

For example, a line with format **iiif** has two integer columns followed by one float column.

### 7.4 Merging and aggregation

The TSF library provides a merge operation that joins lines from multiple files on a specified key column and aggregates the data columns. Available aggregation operators include sum, min, max, and others. The merged output can be emitted as a TSF file or as a two-dimensional table (rows = tags, columns = runs) suitable for import into R or Python.

The user-facing merge command is:

```
magic2 --mergeTSF run1/genes.tsf run2/genes.tsf -o all_genes.txt
```

### 7.5 Relation to user guide

The TSF format is mentioned briefly in the Magic2 User Guide in the context of output files. The internal structure, type codes, and library API are documented here only, as they are relevant to developers who wish to parse TSF output programmatically or extend the TSF library.

## 8 Known Limitations and Future Work

- GPU acceleration of the extension layer (layer 4) is not yet implemented. The splice-junction and gap model would need to be restructured for efficient GPU execution.
- The `--strand` override flag is specified in the documentation and help text but requires implementation in the alignment strategy selection code.



- The `-i` flag currently accepts comma-separated file lists; the documented behaviour (single file or paired pair only, comma lists refused with a clear error message) requires a code change to enforce.

## Acknowledgements

---

We thank David Landsman and Tom Madden for their support. This work was supported by the Intramural Research Program of the National Library of Medicine, National Institutes of Health.